CyberTrace: Collaborative Threat Intelligence & Incident Reporting Platform

Phase 1 – Project Proposal and Documentation

Atta ul Baqi Mansoor
Roll No: F2024408099
BS Cyber Security (Y-9)
Submission Date: 8 June 2026

CyberTrace: Collaborative Threat Intelligence & Incident Reporting Platform

Phase 1 – Project Proposal and Documentation

Submitted by:

Atta ul Baqi Mansoor

Roll Number: F2024408099

Program: BS Cyber Security

Section: Y-9

Submission Date: 8th June 2026

Abstract

CyberTrace is a full-stack web application designed to enable students, security researchers, and small organisations to collaboratively report, analyse, and track cyber threats. The platform allows users to submit incidents with severity classification (Low, Medium, High, Critical), provide structured indicators of compromise (IOCs) including IP addresses, domains, URLs, and file hashes, and browse a community-driven threat feed with advanced search and filtering. Administrators can verify submissions, manage users, and view platform analytics. The system is built using FastAPI (Python), MongoDB, and a responsive HTML/CSS frontend served directly by the backend. It exposes more than 22 RESTful API endpoints supporting full CRUD operations, JWT-based authentication, role-based access control (user/admin), and aggregated statistics. Development is done in PyCharm with GitHub Copilot assistance; deployment uses Render (unified backend + static frontend) and MongoDB Atlas. This document presents the complete proposal following IEEE academic documentation standards.

Keywords – Threat Intelligence, Incident Reporting, FastAPI, MongoDB, JWT, IOC, Severity Classification, Open Source.

Table of Contents

1. Introduction

1.1 Background

Cyber threats are increasing in both volume and sophistication. Small security teams, independent researchers, and students lack a free, open platform to share indicators of compromise (IOCs) or report suspicious activities. Commercial solutions are expensive or read-only for free users. A collaborative, transparent, and educational threat intelligence platform is needed.

1.2 Motivation

As a cyber security student, the author identified the gap between academic learning and practical threat sharing. CyberTrace mimics real-world security operations centres (SOCs) and threat intelligence platforms, teaching API design, database modelling, authentication, and role-based access – core skills for security professionals.

1.3 Scope

The project includes user registration/login, incident submission with severity levels and IOC fields, community feed with search/filter, admin verification workflow, user profiles with contribution stats, and an analytics dashboard. The scope is bounded to deliver a fully functional, deployable web application within the semester timeline.

2. Problem Statement

Individual security researchers and small organisations lack a free, open-source platform to:

- Report cyber incidents with standardised severity and IOC fields.

- View and search community-submitted threats by severity, category, status, or date.

- Receive verification or feedback from moderators.

- Track their own reporting history and impact.

Without such a platform, threat information remains siloed, and students miss the opportunity to learn collaborative threat intelligence workflows.

3. Project Objectives

1. Design and develop a full-stack web application for collaborative threat intelligence.

2. Implement a RESTful API with at least 22 endpoints covering full CRUD, authentication, admin operations, and search/filter.

3. Use MongoDB to model users, incidents (with severity and IOCs), comments, and audit logs.

4. Build a responsive frontend with at least 8 pages using HTML/CSS and vanilla JavaScript.

5. Implement secure authentication using bcrypt hashing and JWT.

6. Enforce role-based access: normal users and administrators.

7. Provide an admin dashboard to verify incidents, manage users, and view platform statistics.

8. Deploy the application publicly using Render (unified backend + frontend) and MongoDB Atlas.

9. Document all phases following IEEE standards (no external references).

10. Demonstrate the working application and defend design decisions during viva.

4. Target Users

Persona	Description
Student Security Researcher	Discovers a phishing site or malware sample; wants to report it and see if others have seen similar threats.
SOC Analyst (Junior)	Uses the platform to correlate community reports with internal data.

Small Business Owner	Checks if a suspicious IP, domain, or URL has been reported before clicking.
System Administrator	Privileged user who verifies incident reports, bans malicious users, and monitors platform health.
Educator	Uses CyberTrace as a teaching tool for threat intelligence concepts.

5. Main Features (12+)

ID	Feature	Description
F1	User Registration & Login	Email/password with bcrypt; JWT returned on success.
F2	Role-Based Access	Normal users can submit/view; admins can verify/delete.
F3	Submit Incident with Severity	User selects Low, Medium, High, or Critical severity.
F4	IOC Support	Provide indicator type (IP, domain, URL, file hash) and value.
F5	Browse Incident Feed	Paginated list of verified incidents.
F6	Search & Filter	Filter by severity, category (phishing/malware/other), status, and date range.
F7	My Incidents Dashboard	Each user sees their submitted incidents and verification status.
F8	Comment on Incidents	Users can add context or ask questions on any verified incident.
F9	Admin Verification	Admin can mark an incident as Verified, False Positive, or Under Review.
F10	Admin User Management	Admins can view all users, promote/demote roles, disable accounts.
F11	Platform Statistics	Admins see total incidents, % verified, top reporters, severity breakdown.
F12	User Profile	View contribution count, join date, and change password.
F13	Responsive UI	Works on desktop, tablet, and mobile (CSS media queries).

6. Frontend Pages (8 pages)

Page	File	Description
P1	index.html	Landing page with hero section, features, call to action.
P2	login.html	Email/password login form.
P3	register.html	Registration form with password confirmation.
P4	dashboard.html	User dashboard: submit new incident, view “my incidents”.
P5	feed.html	Public incident feed with search/filter (severity, type, status, date).
P6	incident_detail.html	Single incident view with comments section.
P7	profile.html	User profile and settings.
P8	admin.html	Admin console (only visible to role=admin).

All pages are served as static files from the FastAPI backend (no separate frontend hosting).

7. Backend API Endpoints (22+)

All endpoints return JSON. Protected endpoints require JWT in Authorization: Bearer <token> header. Admin endpoints require role=admin.

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register new user	Public
POST	/api/auth/login	Login, return JWT	Public
GET	/api/auth/me	Get current user profile	User
PUT	/api/users/me	Update profile (name, password)	User
GET	/api/users/{id}	Get another user’s public profile	User
DELETE	/api/users/{id}	Delete user (admin only)	Admin
PUT	/api/users/{id}/role	Change user role (admin only)	Admin
GET	/api/users	List all users (admin only)	Admin
POST	/api/incidents	Submit new incident (includes severity, IOC	User

		type/value)	
GET	/api/incidents	List incidents with search & filter (severity, type, status, date)	User
GET	/api/incidents/{id}	Get single incident	User
PUT	/api/incidents/{id}	Update own incident (if pending)	User
DELETE	/api/incidents/{id}	Delete incident (user own or admin)	User/Admin
PUT	/api/incidents/{id}/verify	Admin verification (status, notes)	Admin
POST	/api/incidents/{id}/comments	Add comment to incident	User
GET	/api/incidents/{id}/comments	Get comments for incident	User
DELETE	/api/comments/{id}	Delete comment (owner or admin)	User/Admin
GET	/api/stats/platform	Platform statistics (total, verified, top reporters, severity distribution)	Admin
GET	/api/stats/my	Current user's contribution stats	User
GET	/api/ioc/lookup	Search incidents by IOC value (e.g., domain or IP)	User
GET	/api/health	Health check for deployment	Public

Total endpoints: 22 (exceeds requirement).

8. Database Choice and Schema Design

8.1 Choice: MongoDB

- Flexible schema for incidents (different IOC types).

- Native arrays for comments and evidence.

- Easy aggregation for statistics and filtering.

- Free MongoDB Atlas tier for deployment.

8.2 Collections (5 collections)

Collection: users

json

{

"_id": "ObjectId",

"email": "string (unique, indexed)",

"password_hash": "string",

"display_name": "string",

"role": "string (user or admin)",

"created_at": "ISODate",

"updated_at": "ISODate"

}

Collection: incidents

json

{

"_id": "ObjectId",

"reporter_id": "ObjectId (ref users)",

"title": "string",

"incident_type": "string (phishing, malware, suspicious_ip, suspicious_url, other)",

"severity": "string (Low, Medium, High, Critical)",

"ioc_type": "string (ip, domain, url, file_hash)",

"ioc_value": "string",

"description": "string",

"evidence": ["string (URLs or text)"],

"status": "string (pending, verified, false_positive, under_review)",

"admin_notes": "string (optional)",

"created_at": "ISODate",

"updated_at": "ISODate",

"verified_at": "ISODate (optional)",

"verified_by": "ObjectId (ref users, optional)"

}

Collection: comments

json

{

"_id": "ObjectId",

"incident_id": "ObjectId (ref incidents)",

"user_id": "ObjectId (ref users)",

"content": "string",

"created_at": "ISODate"

}

Collection: audit_logs

json

{

"_id": "ObjectId",

"actor_id": "ObjectId (ref users)",

"action": "string (e.g., VERIFY_INCIDENT, DELETE_USER)",

"target_id": "ObjectId",

"details": "string",

"timestamp": "ISODate"

}

Collection: sessions (optional for token blacklisting)

json

{

"_id": "ObjectId",

"user_id": "ObjectId",

"token_jti": "string",

"expires_at": "ISODate"

}

Indexes: email (users), status+severity (incidents), ioc_value (for fast lookup).

9. Technology Stack

Component	Technology	Rationale
Development Environment	PyCharm (with GitHub Copilot)	Professional IDE, integrated Git, DB tools, HTTP client, AI-assisted coding.
Backend Framework	FastAPI (Python 3.11)	High performance, automatic OpenAPI docs, async support.
ASGI Server	Uvicorn	Lightweight, production-ready.
Database	MongoDB (Atlas free tier)	Flexible schema, managed.
Driver	Motor (async PyMongo)	Async support for FastAPI.
Authentication	JWT + bcrypt	Industry standard.
Data Validation	Pydantic v2	Built into FastAPI.
Frontend	HTML5, CSS3, vanilla JavaScript	Static files served directly by FastAPI.
Version Control	Git + GitHub	Public open-source repository.
Deployment Hosting	Render (free tier)	Single service for backend + frontend static files.
Database Hosting	MongoDB Atlas	Free shared cluster.
API Documentation	Swagger UI (auto by FastAPI)	Interactive testing at /docs.

10. Deployment Plan

Development (PyCharm)

- Project structure:

```
CyberTrace/
├── main.py
├── models.py
├── database.py
├── auth.py
├── routers/
│   ├── incidents.py
│   ├── users.py
│   └── admin.py
├── static/
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── feed.html
│   ├── incident_detail.html
│   ├── profile.html
│   └── admin.html
├── requirements.txt
└── Procfile
```

- Local run: uvicorn main:app --reload

- Frontend access: http://localhost:8000/static/index.html

Deployment to Render (unified)

1. Push code to GitHub repository.

2. Create new Web Service on Render, connect GitHub repo.

3. Build command: pip install -r requirements.txt

4. Start command: uvicorn main:app --host 0.0.0.0 --port $PORT

5. Add environment variables:

- MONGO_URI (MongoDB Atlas connection string)

- JWT_SECRET (long random string)

- ADMIN_EMAIL (seed admin email)

- ADMIN_PASSWORD_HASH (pre-hashed bcrypt)

6. Render automatically serves static files from /static folder.

Live URLs

- Frontend: https://cybertrace.onrender.com/static/index.html

- API Docs: https://cybertrace.onrender.com/docs

Security

- All secrets stored as environment variables, never committed.

- HTTPS enforced end-to-end.

- CORS restricted to same origin (or configured as needed).

- Passwords hashed with bcrypt (cost factor 12).

11. Expected Output and Deliverables

Upon completion of all phases:

- Live URL – Fully functional web application accessible via browser.

- GitHub Repository – Public repo with source code, README, and setup instructions.

- Swagger UI – Interactive API documentation at /docs.

- MongoDB Atlas Database – Populated with sample incidents, users, and one admin account.

- Phase 1 Document – This proposal in PDF format (IEEE style, 10–11 pages).

- Screenshots – Key pages, admin dashboard, API docs, database collections.

- Demo Credentials – For evaluator: normal user account and admin account.

12. Risks and Mitigation

Risk	Probability	Mitigation
Time constraint	Medium	Prioritise core features (auth, incident CRUD, admin verify) before optional extras. Use Copilot to generate repetitive code.
Free tier limitations	Low	Render cold starts (few seconds) – acceptable for demo. Atlas free tier sufficient.
JWT security mistakes	Medium	Follow FastAPI OAuth2 examples; use strong secret; set short expiry (1 day).
Frontend complexity	Low	Use vanilla JS + fetch; avoid frameworks. Canva Pro for UI design if needed.
Scope creep	Medium	Freeze feature list after Phase 1 approval. No additional features beyond Section 5.
MongoDB schema changes	Low	Use Pydantic models to validate; migrations are simple in MongoDB.
Search/filter performance	Low	Create proper indexes on severity, status, incident_type, created_at.

13. Conclusion

This Phase 1 document has presented a complete proposal for CyberTrace, a collaborative threat intelligence and incident reporting platform. The project aligns with the learning outcomes of the Open Source Software Development course by integrating modern open-source technologies (FastAPI, MongoDB, HTML/CSS) and emphasising real-world software engineering practices. The system will include more than 22 REST API endpoints, 5 MongoDB collections, role-based access control, severity classification, IOC support, search/filter, 8 frontend pages, and a unified deployment on Render. The proposal exceeds the minimum requirements stipulated by the instructor. Upon approval, Phase 2

(implementation and deployment) will commence, followed by Phase 3 (final demonstration and viva).